KARTU REFERENSI

Kartu Referensi Pola Microservices

Training Arsitektur Microservices dan Cloud dengan Golang

DISIAPKAN UNTUK

Peserta In-house Training

Training	**26 – 29 Oktober 2026**
Isi	**Kapan setiap pola layak digunakan, beserta beban dan risikonya**
Studi kasus	**Sistem simulasi perbankan: account, transfer, notification**
Repositori	**github.com/artivisi/training-microservices-2026-01**

PT ArtiVisi Intermedia
endy@artivisi.com · artivisi.com

Oktober 2026

1 Peta *Concern* Microservices

Setiap sistem microservices harus menjawab sepuluh *concern* berikut. Setiap *concern* ditangani di kode, yaitu *library* yang dipasang di setiap service, atau oleh platform (Kubernetes dan infrastruktur pendukung). Platform Spring Cloud menangani banyak *concern* di *library* ekosistem Java: Eureka untuk *service discovery*, Spring Cloud LoadBalancer (dulu Ribbon) untuk *load balancing* di sisi klien, Resilience4j (dulu Hystrix) untuk *circuit breaker*, dan Config Server untuk konfigurasi. Dukungan ini hanya berlaku untuk service JVM. Platform Kubernetes menyediakan *service discovery*, *load balancing*, konfigurasi, *restart*, dan *rollout* untuk service dalam bahasa apa pun. Service *standalone* di VM, misalnya beberapa service Laravel atau Express.js, tidak memiliki keduanya, sehingga tim menyediakan sendiri setiap *concern* dengan Nginx, `.env`, skrip *deploy*, dan PM2 atau supervisor. Kebijakan *resiliency*, konsistensi data, pemeriksaan *security*, dan instrumentasi tetap ditulis di kode. Service Go di studi kasus mengandalkan platform dan beberapa *library* kecil, tanpa *framework* microservices.

	Concern	**Pertanyaan yang Dijawab**	**Pola dan Teknik di Studi Kasus**	**Di Kode (*Library*)**	**Di Platform atau Infrastruktur**	**Lab**
1	*Decomposition* dan kepemilikan data	*Service boundary* dan pemilik data	Service per domain bisnis, *database per service*, data milik service lain diambil melalui API atau *event*	—	*Database per service*	1, 2
2	Komunikasi sinkron dan asinkron	REST, gRPC, *messaging*	REST untuk *channel*, gRPC untuk panggilan internal, Kafka untuk *event*	`net/http`, grpc-go, kafka-go	Kafka	2, 4, 5
3	*Resiliency*	Tetap berfungsi saat dependensi lambat atau mati	*Timeout*, *retry* dengan *backoff*, *circuit breaker*	failsafe-go	*Probe*, *restart*, *rolling update*	3, 4, 9
4	Konsistensi data	Perubahan data lintas database	*Saga* dengan kompensasi, *transactional outbox*, *idempotency key*, *idempotent consumer*	Kode service (pgx, kafka-go)	—	5, 6
5	*Security*	Siapa boleh memanggil apa	OAuth2 *client credentials*, validasi JWT	JWT *middleware* (REST) dan *interceptor* (gRPC)	Hydra (*authorization server*), API *gateway*	7
6	*Observability*	Kondisi tiap service dan integrasi antarservice	*Log* terstruktur dengan *correlation ID*, metrik, *distributed tracing*	`slog`, Prometheus *client*, OTel SDK	Grafana, Loki, Tempo, Prometheus	8
7	*Configuration* (termasuk *centralized config* dan *secret*)	Nilai yang berbeda per *environment*	*Environment variable* wajib tanpa nilai *default* (12-factor)	`LoadConfig`	ConfigMap, Secret, *secret manager*	1, 9
8	*Service discovery* dan *load balancing*	Alamat service tujuan	Alamat service dibaca dari konfigurasi	— (alamat dari konfigurasi)	Kubernetes Service (DNS), Ingress	9
9	*Packaging* dan *deployment*	Cara service dijalankan dan diperbarui	*Multi-stage* Dockerfile, *image* statis *non-root*, *rolling update*	Dockerfile	Kubernetes Deployment, *probe*, *rolling update*	9
10	*Automation* (CI/CD)	Dari *commit* ke *production*	*Pipeline* per service: *test*, *build image*, *deploy*	Target `Makefile`	*Pipeline* (git server dan runner CI, Jenkins)	10

2 Cara Membaca

Setiap baris memuat satu pola. Kolom **Layak Digunakan Bila** dan **Tidak Perlu Bila** menjadi dasar keputusan. Kolom **Beban dan Risiko** mencatat pekerjaan tambahan yang muncul setelah pola diterapkan. Kolom **Lab** menunjuk lab studi kasus yang menerapkan pola tersebut. *Checkpoint* setiap lab tersedia sebagai *tag* `lab-NN` di repositori studi kasus.

Setiap pola menambah komponen yang harus dioperasikan. Pola yang tidak menjawab masalah yang sedang dihadapi sistem hanya menambah beban tanpa manfaat.

3 Dekomposisi dan Komunikasi

Pola	Masalah yang Diselesaikan	Layak Digunakan Bila	Tidak Perlu Bila	Beban dan Risiko	Lab
Dekomposisi service dan *database per service*	Tim dan fitur saling menunggu karena satu *codebase* dan satu *database* digunakan bersama. Perubahan skema untuk satu fitur ikut memengaruhi fitur lain	Batas domain bisnis sudah jelas (rekening, transfer, notifikasi). Tiap bagian memiliki tim, jadwal rilis, atau kebutuhan skala yang berbeda	Satu tim kecil mengelola seluruh aplikasi. Batas domain masih sering berubah. Pemisahan modul di dalam satu aplikasi (*modular monolith*) sudah cukup	*Join* lintas *database* tidak tersedia, sehingga data diambil melalui API atau *event*. Data antarservice hanya mencapai *eventual consistency*. Jumlah *deployment* dan *database* yang dioperasikan bertambah	1, 2
REST sinkron	Pemanggil membutuhkan jawaban saat itu juga, misalnya data rekening sebelum transfer diproses	API untuk kanal (*mobile*, web, mitra). Klien beragam dan interoperabilitas lebih penting dibandingkan efisiensi	Panggilan internal yang sering dengan kontrak ketat (gunakan gRPC). Pemanggil tidak menunggu hasil (gunakan *messaging*)	Pemanggil ikut lambat atau gagal saat service tujuan bermasalah. Kontrak hanya tercatat di dokumentasi dan tidak diperiksa *compiler*	1, 2, 3
gRPC	Kontrak antarservice internal tidak terdefinisi tegas. Serialisasi JSON boros untuk panggilan internal yang sering	Panggilan antarservice internal. Kontrak perlu diperiksa saat kompilasi (protobuf). Autentikasi, log, dan *resiliency* ditangani terpusat di *interceptor*	Klien berupa *browser* atau mitra eksternal. Tim belum siap mengelola protoc dan versi kontrak	Kode hasil *generate* harus dikelola. Perubahan kontrak wajib *backward compatible*, nomor *field* tidak boleh digunakan ulang. *Debugging* dengan grpcurl membutuhkan file `.proto` atau *reflection* yang aktif di server. HTTP/2 menggunakan *persistent connection*, sehingga *Service* Kubernetes (L4) mengarahkan semua panggilan ke satu *pod*. Penyebaran panggilan ke semua *pod* membutuhkan *load balancing* di sisi klien atau *proxy* L7	4
***Messaging* asinkron** (Kafka)	Pengirim bergantung pada ketersediaan semua penerima. Satu *event* bisnis perlu diproses beberapa service	Penerima tidak perlu menjawab saat itu juga (notifikasi, audit, analitik). Beberapa *consumer* memproses *event* yang sama. Lonjakan beban perlu diredam	Pemanggil membutuhkan hasil untuk melanjutkan proses. Volume rendah dengan satu penerima saja	Pengiriman *at-least-once* menghasilkan duplikat. Urutan hanya terjamin di dalam satu *partition* (per *key*). Broker menjadi infrastruktur tambahan. Alur sulit ditelusuri tanpa *tracing*	5, 6

4 *Resiliency*

Pola	Masalah yang Diselesaikan	Layak Digunakan Bila	Tidak Perlu Bila	Beban dan Risiko	Lab
Timeout	Service tujuan yang lambat menahan *goroutine*, koneksi, dan *request* pemanggil tanpa batas waktu	Setiap panggilan jaringan, tanpa kecuali. Nilainya ditentukan dari latensi p99 service tujuan dan *deadline* yang dijanjikan ke pemanggil	Tidak ada pengecualian untuk panggilan jaringan	Timeout yang terlalu pendek menggagalkan *request* yang sebenarnya akan berhasil. Setelah timeout, pemanggil tidak tahu status di sisi tujuan, termasuk apakah debit sudah tercatat	3, 4
Retry dengan *backoff*	Gangguan sesaat (koneksi ditolak, HTTP 503, *pod restart*) menggagalkan *request* yang akan berhasil pada percobaan berikutnya	Gangguan bersifat sementara, dan operasi *idempotent* atau *request* terbukti belum diproses (koneksi ditolak). HTTP 503 dan `Unavailable` hanya aman di-retry bila server menjamin status tersebut dikirim sebelum operasi dijalankan (di studi kasus: *fault injection* di account). Bila tidak ada jaminan tersebut, operasi harus dibuat *idempotent* lebih dahulu (Lab 5)	Operasi tidak *idempotent* setelah timeout (risiko debit ganda). *Error* bisnis (4xx, saldo tidak cukup). *Circuit breaker* sedang *open*	Retry tanpa *backoff* dan *jitter* memperberat service yang sedang bermasalah (*retry storm*). Retry di beberapa tingkat panggilan membuat jumlah percobaan berlipat ganda. Latensi total bertambah	3, 4, 5
Circuit breaker	Pemanggil terus mengirim *request* ke service yang sedang gagal. *Request* yang tertahan menumpuk dan kegagalan merambat ke pemanggil	Ketergantungan sinkron ke service lain yang dapat gagal dalam waktu lama. *Fail fast* dengan status jelas lebih baik dibandingkan menunggu. HTTP 503 hanya digunakan bila belum ada perubahan yang diterapkan, sehingga pemanggil aman mengulang	*Database* milik service sendiri (timeout dan *connection pool* sudah cukup). Volume panggilan terlalu rendah sehingga ambang kegagalan tidak bermakna. Langkah kompensasi saga, karena *breaker* yang baru berpindah ke *open* justru menolak *reversal*. Untuk *reversal*, gunakan retry berbatas waktu tanpa *breaker* (Lab 5)	Ambang dan *delay* perlu disetel berdasarkan data. Setiap *pod* memiliki *breaker* sendiri dengan state berbeda. Perubahan state wajib tercatat di log dan metrik	3, 4, 8

5 Konsistensi Data

Pola	Masalah yang Diselesaikan	Layak Digunakan Bila	Tidak Perlu Bila	Beban dan Risiko	Lab
Saga	Satu transaksi bisnis mengubah data di beberapa *database*. *Two-phase commit* membuat transaksi bergantung pada ketersediaan semua peserta sekaligus	Proses bisnis lintas service dengan aksi kompensasi untuk setiap langkah (debit dibatalkan dengan kredit balik)	Seluruh data berada dalam satu *database* (transaksi lokal sudah cukup)	State antara (`DEBITED`) terlihat oleh pembaca. Kompensasi dapat gagal dan membutuhkan penanganan manual (`COMPENSATION_FAILED`). Setiap langkah dan kompensasi harus *idempotent*. *Graceful shutdown* dan *grace period* platform harus mencakup durasi terburuk satu saga. Proses yang dihentikan paksa di antara langkah meninggalkan state antara, dan *recovery job* harus menyelesaikannya. Langkah yang tidak dapat dikompensasi (misalnya pengiriman dana ke bank lain) diletakkan paling akhir (*pivot transaction*), dan langkah sesudahnya wajib dapat di-retry hingga berhasil	5
Transactional outbox	Penulisan ke *database* dan *publish* ke broker terdiri dari dua operasi terpisah. *Crash* di antara keduanya menghasilkan perubahan data tanpa *event*, atau *event* tanpa perubahan data	Perubahan state wajib diikuti *event* yang tidak boleh hilang (status transfer, mutasi rekening)	*Event* bersifat informatif dan boleh hilang sesekali. Service tidak memiliki *database* sendiri	Tabel outbox terus bertambah dan perlu dibersihkan. *Relay* menambah latensi sebesar interval *polling*. *Crash* setelah *publish* dan sebelum baris outbox ditandai terkirim menghasilkan duplikat. Menjalankan lebih dari satu *relay* sekaligus membutuhkan `FOR UPDATE SKIP LOCKED`	5
Idempotency key	Klien atau retry mengulang *request* (timeout, tombol ditekan dua kali), dan server memprosesnya dua kali	Operasi pengubah data yang dapat diulang oleh klien atau retry: transfer, debit, kredit, pembayaran	Operasi baca. Operasi yang *idempotent* secara alami (PUT dengan nilai absolut, DELETE)	*Key* disimpan dengan *unique constraint*. *Request* dengan *key* sama dan isi berbeda perlu aturan sendiri (HTTP 422). *Request* ulang dijawab dengan kode HTTP yang sama seperti respons pertama, sehingga penyebab kegagalan harus tersimpan. Masa simpan *key* harus ditetapkan	5, 6
Idempotent consumer	Broker mengirim ulang *message* (*rebalance*, *crash* sebelum *commit offset*, *replay*) sehingga efeknya terjadi dua kali	Setiap *consumer* dengan pengiriman *at-least-once* yang efeknya tidak boleh ganda (mutasi saldo, notifikasi ke nasabah)	Efek *consumer* sudah *idempotent* secara alami (*upsert* nilai terbaru, isi *cache*)	ID *event* wajib ditulis dalam transaksi yang sama dengan efeknya. Tabel `processed_event` perlu kebijakan retensi. Efek ke sistem luar (SMS, email) tidak dapat ikut transaksi *database*. *Event* yang tidak mungkin diproses (*poison message*) dilewati dengan log `ERROR` atau dikirim ke *dead-letter topic*, agar *partition* tidak tertahan	6

5.1 Perbandingan *Orchestration* dan *Choreography* pada Saga

Aspek	*Orchestration*	*Choreography*
Pengendali alur	Satu service (orkestrator) memanggil setiap langkah dan menyimpan state saga	Setiap service bereaksi terhadap *event* service lain, tanpa pengendali pusat
Visibilitas	State saga dapat dibaca di satu tabel orkestrator	State tersebar, hanya terlihat melalui *tracing* dan log *event*
Ketergantungan	Orkestrator mengenal semua peserta	Peserta hanya mengenal *event*, tetapi alur bisnis tersirat di banyak service
Cocok untuk	Alur dengan urutan ketat dan kompensasi bertingkat, seperti transfer dana	Alur pendek dengan sedikit peserta dan langkah yang saling lepas
Risiko	Orkestrator menampung terlalu banyak logika bisnis	Siklus *event* antarservice dan alur yang sulit diubah tanpa menyentuh semua peserta
Di studi kasus	Transfer sebagai orkestrator: `PENDING` → `DEBITED` → `COMPLETED`, atau `COMPENSATED` / `FAILED` (Lab 5)	Tidak diterapkan. Notification hanya *consumer event* akhir transfer dan tidak menjadi langkah saga. Versi *choreography*: account mem-*publish* `AccountDebited`, lalu service lain bereaksi terhadap *event* tersebut dan melakukan kredit tanpa orkestrator

6 Security dan Observability

Pola	Masalah yang Diselesaikan	Layak Digunakan Bila	Tidak Perlu Bila	Beban dan Risiko	Lab
OAuth2 client credentials dan JWT	Service tujuan tidak mengetahui identitas dan hak pemanggil. Jaringan internal dianggap aman tanpa verifikasi	Panggilan service ke service dan kanal ke service yang membutuhkan identitas dan *scope*. Token divalidasi lokal dengan JWKS tanpa memanggil *authorization server* di setiap *request*	Identitas pemanggil sudah dijamin mTLS dari *service mesh* dan otorisasi per *scope* tidak dibutuhkan	*Authorization server* menjadi ketergantungan kritis (token dan JWKS). JWT tidak dapat dicabut sebelum kedaluwarsa, sehingga masa berlakunya harus pendek. Token tanpa TLS dapat disadap	7
Correlation ID dan *distributed tracing*	Satu transaksi melewati beberapa service dan broker. Log tiap service tidak terhubung, sehingga sumber kelambatan atau kegagalan sulit ditemukan	Sejak sistem memiliki dua service. Konteks diteruskan melalui *header* HTTP, *metadata* gRPC, dan *header* Kafka	Aplikasi tunggal tanpa panggilan ke service lain (log biasa sudah cukup)	Setiap titik masuk dan keluar wajib meneruskan konteks. Satu titik yang terlewat memutus *trace*. Volume data *trace* besar sehingga perlu *sampling*. *Baggage* dari pemanggil luar tidak diteruskan ke service internal	8
Metrics (Prometheus)	Laju *error*, latensi, state *breaker*, dan jumlah *event* outbox yang belum terkirim baru diketahui setelah pengguna mengeluh	Setiap service: *rate*, *error*, *duration* per *route*, ditambah metrik bisnis (`transfers_total`) dan metrik pola (`circuit_breaker_state`, `outbox_pending`)	Selalu diperlukan. Jumlah dan isi label harus dibatasi	Label berkardinalitas tinggi (ID transfer, nomor rekening) membebani Prometheus. Metrik tanpa *alert* dan *dashboard* tidak pernah dibaca	8
Health probe: *liveness* dan *readiness*	Kubernetes mengirim trafik ke *pod* yang belum siap, atau membiarkan *pod* yang macet tetap berjalan	Setiap service di Kubernetes atau di belakang *load balancer*. *Liveness* (`/healthz`): proses masih merespons. *Readiness* (`/readyz`): *dependency* wajib seperti *database* dapat dijangkau	*Liveness* tidak perlu memeriksa *dependency* eksternal. *Liveness* yang ikut memeriksa *database* memicu *restart* massal saat *database* mati, padahal *restart* tidak memperbaiki apa pun	Ambang atau timeout *probe* yang terlalu ketat memicu *restart* berulang. *Readiness* yang memeriksa service lain menghasilkan kegagalan berantai. *Health service* gRPC yang tidak memeriksa *database* tidak cocok sebagai *readiness*	1, 9

7 Deployment dan Operasi

Pola	Masalah yang Diselesaikan	Layak Digunakan Bila	Tidak Perlu Bila	Beban dan Risiko	Lab
12-factor config	Konfigurasi tertanam di kode atau *image*, sehingga *image* yang sama tidak dapat dipindahkan antar-*environment*	Service yang di-*deploy* ke lebih dari satu *environment*. Nilai rahasia dipisahkan ke Secret, nilai lain ke ConfigMap	Konstanta bisnis yang sama di semua *environment* tetap ditulis di kode	Variabel wajib yang hilang harus menghentikan *startup* dengan pesan *error* yang menyebut nama variabelnya. Nilai *default* menyembunyikan konfigurasi yang terlewat hingga *production*	1, 9
Container image: *multi-stage*, *static*, *non-root*	*Image* yang memuat *compiler* dan *shell* berukuran besar dan memperluas *attack surface*. Proses *root* ditolak platform dengan kebijakan ketat (OpenShift *restricted SCC*)	Setiap service yang berjalan di Kubernetes atau OpenShift. Go dengan `CGO_ENABLED=0` menghasilkan *binary* statis yang cukup dijalankan di `FROM scratch`	Aplikasi membutuhkan *library* C (CGO) atau *tools* di dalam *container*. Gunakan *base image* minimal sebagai pengganti `scratch`	*Shell* untuk *debugging* tidak tersedia (gunakan `kubectl debug`). Sertifikat CA harus disalin manual. Aplikasi wajib berjalan dengan UID acak dan *filesystem read-only*	9
Rolling update	Mengganti semua *pod* sekaligus menghentikan layanan selama versi baru dijalankan	Versi baru kompatibel dengan versi lama selama masa transisi, baik API maupun skema *database*	Perubahan skema yang tidak kompatibel: gunakan migrasi *expand/contract* (kolom baru ditambahkan lebih dahulu, kolom lama dihapus setelah versi lama berhenti), atau strategi *recreate* dengan *downtime*. API yang tidak kompatibel: gunakan *blue-green*, karena trafik dipindahkan sekaligus	Dua versi melayani trafik bersamaan. *Readiness* yang keliru meloloskan *pod* yang rusak. *Rollback* membutuhkan *tag image* yang tidak berubah, sehingga *tag* `latest` harus dihindari. *Build* ulang dengan *tag* yang sama juga tidak memicu *rollout*. *Pod* yang dihentikan masih menerima *request* beberapa detik (`preStop` *sleep*), dan `terminationGracePeriodSeconds` harus mencakup batas *shutdown* service	9, 10
Horizontal Pod Autoscaler	Beban berfluktuasi. Dengan jumlah replika yang tetap, kapasitas berlebih saat sepi dan tidak mencukupi saat ramai	Service *stateless* dengan beban naik turun dan metrik yang sejalan dengan beban (CPU, *request* per detik)	*Bottleneck* berada di *database* atau service lain, sehingga tambahan replika hanya menambah koneksi. *Consumer* Kafka melebihi jumlah *partition*. Beban stabil	*Resource requests* harus akurat. *Scale-up* membutuhkan waktu (*startup* dan *readiness*). Batas koneksi *database* harus menampung replika maksimum	Konsep (hari 4)
CI/CD per service	Rilis satu service menunggu service lain. *Deployment* manual tidak seragam dan tidak tercatat	Service dirilis independen. Setiap *commit* diuji, dijadikan *image* dengan *tag* berupa *commit hash*, lalu di-*deploy* otomatis. *Pipeline* hanya memanggil target `make`, sehingga dapat dipetakan ke Jenkinsfile	Service selalu dirilis bersamaan. Kondisi ini menandakan *distributed monolith*, sehingga *service boundary* perlu diperbaiki lebih dahulu	Jumlah *pipeline* sebanding dengan jumlah service. Perubahan modul bersama (`proto`) memicu *pipeline* beberapa service. *Credential registry* dan *cluster* harus dikelola di *runner*	10
GitOps	State *cluster* menyimpang dari *manifest* di repositori. Perubahan langsung di *cluster* tidak tercatat	Banyak *cluster* atau *environment*. Audit perubahan wajib. *Pipeline* tidak boleh menyimpan *credential cluster*	Satu *cluster* kecil dengan *pipeline push* yang sudah tercatat. Tim belum terbiasa dengan *manifest* dan CI	Komponen tambahan (*controller* di dalam *cluster*) dan repositori *manifest* terpisah. Secret membutuhkan mekanisme enkripsi tersendiri	Konsep (hari 4)

8 Gejala *Distributed Monolith*

Sistem yang sudah dipecah menjadi beberapa service tetapi masih harus diubah, diuji, dan dirilis bersamaan memikul beban *microservices* tanpa mendapatkan manfaatnya. Tiga gejala atau lebih yang tercentang menandakan *service boundary* perlu ditinjau ulang.

	Gejala	Penyebab Umum	Arah Perbaikan
☐	Fitur baru hampir selalu mengubah beberapa service sekaligus	*Service boundary* mengikuti *layer* teknis (API, logika, data) dan tidak mengikuti domain bisnis	Tetapkan ulang *service boundary* menurut domain. Gabungkan service yang selalu berubah bersamaan
☐	Service harus dirilis bersamaan dengan urutan tertentu	Kontrak API berubah tanpa *backward compatibility*	Perubahan kontrak bersifat aditif. Versi kontrak lama tetap dilayani selama masa transisi
☐	Beberapa service membaca atau menulis tabel yang sama	*Database* bersama dipertahankan setelah pemecahan service	*Database per service*. Data milik service lain diambil melalui API atau *event*
☐	Satu service mati menghentikan hampir semua alur	Rantai panggilan sinkron panjang tanpa timeout dan *circuit breaker*	Timeout dan *breaker* di setiap panggilan. Langkah yang tidak perlu ditunggu dipindahkan ke *messaging*
☐	Satu *request* dari kanal memicu panggilan bolak-balik antarservice (A → B → A)	Data dan logika yang saling membutuhkan ditempatkan di service berbeda	Gabungkan logika tersebut ke satu service, atau salin data yang dibutuhkan melalui *event*
☐	*Library* bersama memuat model domain dan logika bisnis	Kode bersama digunakan untuk menghindari duplikasi lintas service	*Library* bersama hanya berisi kode teknis (log, *tracing*, kontrak protobuf). Logika bisnis tetap di service pemiliknya
☐	Satu service tidak dapat diuji tanpa menjalankan semua service lain	Ketergantungan sinkron yang tidak diisolasi di kode	Klien service lain dibungkus *interface* agar dapat diganti saat pengujian dengan `httptest` atau `bufconn`
☐	Seluruh service berbagi satu *pipeline* dan satu nomor versi	Repositori dan proses rilis tetap dikelola sebagai satu aplikasi	*Pipeline* per service dengan filter *path*. *Tag image* per service